Draven Brown
Data 219
5/6/26

Data

For this project, I used an NBA player-season dataset from the 1996-2022 seasons. Each row represents one player in one season, not one unique player. The original dataset had 12,844 player-season rows. After cleaning and filtering the data to only include players with more than 50 games played in a season, my array had 7,615 player-season records. The hash table had 1,531 keys because each key represented one unique player name.

Question I Tried to Answer

My research question was: How does amount of scoring relate to scoring efficiency for NBA players from 1996-2022?

I split player-seasons into three scoring groups, low scorers under 10 points per game, medium scorers from 10 to under 20 points per game, and high scorers over 20 points per game. Then I compared the average true shooting percentage for each group.

Cleaning Steps

-Removed the unnecessary Unnamed 0 column because it was a useless column
- The main missing values were in the college column, which is not always an error because some players did not attend college or did not have a college listed.
- Checked for duplicate rows. Repeated player names were not removed because each row represents a different season.
- Checked crucial stats such as pts and ts_pct were in a possible range. Points should not be negative, and ts_pct should be between 0 and 1.
- Filtered the dataset to only include player-seasons where gp > 50. This reduced outliers from players who barely played and made the analysis more reliable.

Chosen Data Structures and Justification

I used two data structures: an array and a hash table. I chose an array because it is simple and stores every player-season record in order. This makes it easy to loop through the entire dataset and perform calculations. The downside is that searching for one specific player is slower because the program may need to parse through every record.

I chose a hash table because it is better for searching by player name. In the hash table, the player name was the key, and the value was a list of all seasons for that player. This allowed me to quickly find a player like LeBron James without looping through every player-season record. The hash table was useful for direct lookup, but it was not always faster for full analysis because my scoring efficiency analysis still had to go through every season record.

Operations and Implementation

Draven Brown
Data 219
5/6/26

The first operation was searching for one player. In the array, I looped through every record and checked if player_name matched the player I wanted. In the hash table, I used the player name as a key and directly accessed that player's list of seasons. This compares an O(n) array search against an average O(1) hash table lookup.

The second operation was the main analysis. I looped through the records and placed each player-season into one of three scoring groups based on pts. Then, I manually added up ts_pct and counted the number of records. Then I divided the total ts_pct by the count to get the average true shooting percentage. I implemented this once using the array and once using the hash table.

Results Points

Group	Points Per Game	Count	Average TS%
Low scorers	< 10	4,118	0.5265 = 52.65%
Medium scorers	10 to < 20	2,852	0.5490 = 54.90%
High scorers	> 20	645	0.5670 = 56.70%

The results showed that scoring efficiency increased as scoring volume increased. Low scorers averaged about 52.65% true shooting, medium scorers averaged about 54.90%, and high scorers averaged about 56.70%. This suggests that higher-scoring players were also more efficient scorers on average.

Performance Analysis

Data Structure	Analysis Time	Big O for Main Analysis
Array/List	0.017929 seconds	O(n)
Hash Table/Dictionary	0.021327 seconds	O(n)

For the main scoring efficiency analysis, the array was slightly faster than the hash table. This was because the analysis had to check every player-season record no matter which data structure was used so neither had an advantage. The array was probably faster because it stored all player-seasons in one simple list, while the hash table had to loop through each player key and then through that player's list of seasons.

Conclusion

Overall, my project found that higher-scoring NBA players were more efficient on average in this dataset. The high-scoring group had the highest average true shooting percentage, followed by the medium, and then the low-scoring groups. For data structures, the hash table was the better choice for direct player lookup, but the array was slightly faster for the full analysis.